

AI-агенти в роботі:

Skills, MCP і контекст

Що працює вже сьогодні, що варто спробувати завтра — і чому головна валюта агента це не токени, а увага.

♦ 6 блоків · 24 хв

♦ теорія → приклад → наша практика

♦ S — картки спікера

ТЕЗИ

- Привітання, 15 секунд. Не читати заголовок вголос.
- «Я не буду продавати вам AI. Я покажу механіку: з чого складається агент, чому він тупить і що з цим робити руками.»
- Одразу озвучити формат: питання можна кидати по ходу, але великі — в кінці, на них є 5 хвилин.

ТАЙМІНГ

- Загальний бюджет — 24 хв + 5 хв Q&A. Якщо на 12-й хвилині ви ще в блоці Skills — пропускайте слайди з міткою «опційний».

ЗНАЙОМА КАРТИНА

Агент прочитав 40 файлів і зробив не те

- ◆ Витратив пів контексту на читання того, що йому не було потрібно
- ◆ Проігнорував конвенції проєкту, хоча вони написані
- ◆ На третьому повідомленні «забув» те, про що домовились на першому
- ◆ Зробив руками те, що детерміновано робить скрипт за 200 мс

Майже завжди проблема не в моделі. Проблема в тому, **що саме** ми поклали їй у контекст — і що забули покласти.

1 Skills

Одиниця знання. Процедура, яку агент підвантажує на вимогу.

2 Агенти

Виконавець із власним контекстом і критерієм зупинки.

3 MCP

Доступ до зовнішнього світу + свій сервер-агрегатор.

4 Jev

Нова технологія на горизонті. Дивимось, не впроваджуємо.

5 Токени

Як міряти і як економити, не втрачаючи якість.

6 Контекст проєкту

Граф коду + LLM вікі на 1000+ файлів.

ТЕЗИ

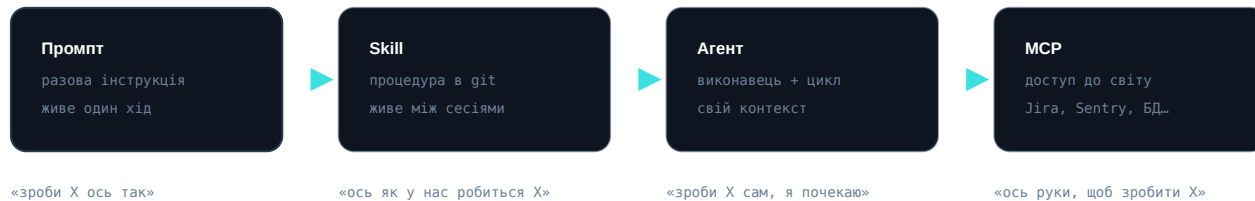
- Почати з болю — попросити підняти руку тих, у кого таке було. Це заходить.
- Головна теза доповіді, яку повторимо ще двічі: **агент — це функція від контексту, а не від промпу.**
- Мапу проговорити швидко, по одному слову на блок. Не читати описи.

ЯКЩО СПИТАЮТЬ

«А модель же стає розумнішою, це саме не вирішиться?» Розмір вікна росте, але деградація якості на довгому контексті (context rot) нікуди не дівається. Чим більше сміття, тим гірше рішення — незалежно від моделі.

Чотири рівні, які постійно плутають

Кожен вирішує свою проблему. Більшість невдач — це спроба закрити задачу не тим рівнем.



ТЕЗИ

- Ключова відмінність промпта і скіла: промпт живе один хід, скіл лежить у git і працює для всієї команди.
- Ключова відмінність скіла й агента: скіл — це *що робити*, агент — це *хто робить і в якому контексті*.
- MCP не робить рішень взагалі. Це руки, не голова.
- Типова помилка: намагатись вирішити промптом те, що має бути скілом; або зробити агента там, де вистачило б скрипта.

Хто за що відповідає

РІВЕНЬ	ВІДПОВІДАЄ ЗА	ДЕ ЖИВЕ	ЧИМ ЛАМАЄТЬСЯ
Модель	Міркування, генерація	провайдер	Шумний контекст
Skill	Процедуру: що робити і в якому порядку	.agents/skills/	Скіл-енциклопедія
Агент	Виконання: свій контекст, своя модель, критерій зупинки	.claude/agents/	Агент-універсал
MCP	Доступ до зовнішніх систем і даних	MCP-сервер	40 тулів у кожній сесії
Hooks і скрипти	Детермінізм: те, що має статись завжди	scripts/ + hooks	Логіка, засунута в модель

Якщо запам'ятати один слайд із доповіді — цей. Далі ми просто розкриваємо кожен рядок.

ТЕЗИ

- Пройти по колонці «чим ламається» — це дзеркало антипатернів, які будуть далі.
- Наголосити на останньому рядку: **детермінізм — не в модель**. Це найдешевша оптимізація, яка існує.
- Сказати, що це слайд, до якого варто повернутись — він є у роздатці.

Skill — це пакет процедури, а не промпт

```
# звичайна папка в репозиторії
release-notes/
├── SKILL.md      ← обов'язковий
├── references/   ← глибокі деталі
│   └── format.md
├── scripts/      ← детермінізм
│   └── collect.mjs
└── assets/       ← шаблони
    └── template.md
```

```
--- SKILL.md ---
name: release-notes
description: Збирає release notes з git-історії
            між тегами. Використовуй на /release-notes,
            «що увійшло в реліз», «зміни з минулого разу».
---
# Release notes
1. Визнач діапазон: git describe --tags --abbrev=0
2. Запусти scripts/collect.mjs ...
```

→ Це не плагін і не код

Агент не «встановлює» скіл. Він бачить його опис і вирішує, чи читати тіло. Формат навмисно мінімальний: два обов'язкові поля frontmatter і Markdown нижче.

◆ Чому не CLAUDE.md / AGENTS.md

Той файл читається **завжди** і платиться в кожній сесії. Скіл читається **лише коли потрібен**. У корені — політики й конвенції, у скілах — процедури.

◆ Чому не просто промпт

Скіл лежить у git: його рев'юють, версіонують, він однаковий у всіх у команді й не залежить від того, хто як сформулював.

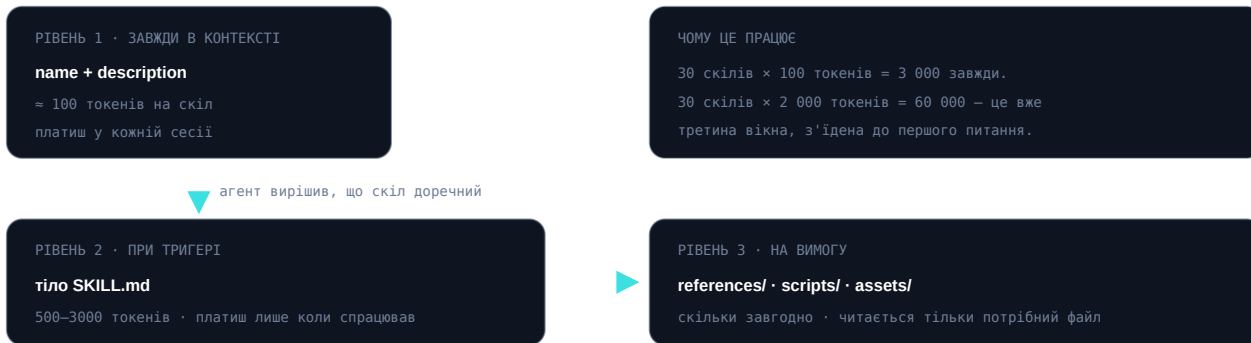
ТЕЗИ

- Провести аналогію: **скіл — це README, написаний для агента, а не для людини**. Імператив, без води.
- Звернути увагу на description у прикладі: там навмисно є і слеш-команда, і живі фрази користувача.
- Наголосити: формат мінімальний, це важливо — на нього легко перейти й важко зламати.

ЯКЩО СПИТАЮТЬ

«А чим це відрізняється від .cursorrules?» Cursor-правила прив'язані до глобів і вантажаться за шляхом файлу. Skill вантажиться за *наміром* — агент сам вирішує, що ця процедура зараз доречна. І він портативний між інструментами, про що буде окремий слайд.

Progressive disclosure: платиш тільки за те, що відкрив



Це той самий lazy loading, до якого ми звикли у фронті. Тільки бюджет тут не кілобайти, а **увага моделі**.

ТЕЗИ

- Це найважливіший слайд блоку. Не поспішати.
- Аналогія з фронтом заходить: «уявіть, що ви імпортуєте весь lodash у кожен компонент».
- Головний висновок для автора скіла: **кожен рядок у SKILL.md — це рядок, який хтось прочитає, навіть якщо він йому не потрібен.**
- Рівень 3 — це те, що більшість не використовує. А саме там має лежати 80% обсягу.

ЯКЩО СПИТАЮТЬ

«Звідки цифра 100 токенів?» Це name + description із frontmatter, які інструмент кладе в системний промпт. Далі на слайді про бюджет буде реальний вимір з нашого репозиторію: 11 скілів + 6 агентів + 2 команди = близько 1 450 токенів описів.

Description — це не опис. Це тригер

Єдине, що агент бачить до того, як вирішити читати скіл. Якщо тут немає слів, якими користувач формулює задачу — скіл не спрацює ніколи.

✗ Так не працює

```
description: Скіл для роботи  
з тестами.
```

Немає тригерних слів. Немає назв інструментів. Немає фраз, якими люди реально просять. Агент не знає, коли це доречно — і не відкриває.



✓ А так працює

```
description: Пише unit-тести  
для composables і класів-моделей  
на Vitest. Використовуй на  
/write-tests, «покрій тестами»,  
«напиши тест на useApi»,  
«додай кейс на цю помилку».
```

Що робить + для чого + якими словами про це просять.

Клади всередину: назви фреймворків (Vitest, Playwright), типи артефактів (junit, trace), дієслова дії (покрій, знайди, проаналізуй).

Клади синоніми й мову команди. У нас у descriptions свідомо є російські й українські формулювання — бо люди так і питають.

Ліміт — 1024 символи. Це багато. Використай їх: це найдешевші токени у всій системі.

ТЕЗИ

- Розповісти історію: скіл написаний ідеально, але не тригериться — і людина вирішує, що «скіли не працюють».
- Правило великого пальця: напиши description і дай колезі. Якщо він не зрозумів, коли це викликається — агент теж не зрозуміє.
- Це єдине місце, де багатослівність корисна. Скрізь інде — навпаки.

ЯКЩО СПИТАЮТЬ

«А можна змусити викликати примусово?» Слеш-командою — так. Але це вже не автоматизація, а ручний виклик. Мета — щоб скіл підхоплювався сам.

Сім правил, які роблять скіл робочим

- 1 Імператив, не наратив.** «Запусти Х. Перевір Y. Якщо Z — зупинись.» Не «зазвичай ми робимо так, бо історично...».
- 2 Одна процедура = один скіл.** Якщо в назві з'явилося «і» — це два скіли.
- 3 ≤ 500 рядків у SKILL.md.** Вилізло більше — значить, частина мала бути в [references/](#).
- 4 Дерева рішень — у references.** Основний файл лишає маршрут, деталі читаються за потреби.
- 5 Детермінізм — у scripts.** Усе, що можна порахувати кодом, має рахувати код. Модель дорога й недетермінована.
- 6 Приклади входу й виходу.** Один хороший приклад економить абзац пояснень і дає модель формату.
- 7 Живе в git.** Рев'ю, версія, історія змін. Скіл, який лежить у когось локально — це не скіл, а нотатка.

! Тест на якість скіла

Дай його новому джуну без пояснень. Якщо він виконає процедуру правильно з першого разу — скіл готовий. Якщо перепитає «а що мається на увазі тут» — саме там модель теж помилиться.

ТЕЗИ

- Правило 5 — найважливіше й найчастіше ігнороване. Приклад: «порахуй, скільки файлів змінилось» — це `git diff --name-only | wc -l`, а не робота моделі.
- Правило 7 має соціальний бік: скіл у git означає, що знання команди перестає бути в голові однієї людини.
- «Тест на джуна» — озвучити обов'язково, це найзапам'ятовуваніша частина слайда.

Збираємо скіл **release-notes** з нуля

Нейтральна задача, яка є в будь-якому проєкті. Питання одне: що куди покласти.

L1 frontmatter

```
name: release-notes
description: Формує
  release notes між тегами
  з conventional commits.
  /release-notes, «що
  увійшло в реліз».
```

~60 токенів. Висить завжди.
Тут тільки «що» і «коли».

L2 SKILL.md

```
## Кроки
1. node scripts/collect.mjs
  → поверне JSON комітів
2. Згрупуй за типом
  (feat / fix / perf)
3. Формат — у
  references/format.md
4. Питай про breaking
  changes перед публікації
```

~250 токенів. Маршрут, не
інструкція з усіма
випадками.

L3 references + scripts

```
references/format.md
  приклади для кожного
  типу, тон, шаблон MR

scripts/collect.mjs
  git log, парсинг,
  фільтр merge-комітів,
  JSON на вихід
```

0 токенів, поки не
знадобились. Скрипт —
взагалі не токени.

Межа проходить так: у SKILL.md — рішення, які має ухвалити модель. У references — довідка, потрібна не завжди. У scripts — усе, де немає рішень, а є обчислення. Якщо крок можна описати словом «порахуй», «знайди», «розпарси» — це скрипт.

ТЕЗИ

- Провести залу по трьох рівнях повільно: «а це ми куди покладемо? а це?»
- Питання до залу (риторичне): «Куди покласти список наших типів комітів?» → references, бо він великий і потрібен не завжди.
- Наголосити фразу-критерій: **«порахуй / знайди / розпарси» = скрипт.**

ЯКЩО СПИТАЮТЬ

«А якщо скрипт складний, це ж теж підтримка?» Так. Але детермінований скрипт ти пишеш і тестуєш один раз, а модель робить це заново в кожній сесії — і кожного разу трохи інакше.

Універсальні скіли: один стандарт, 32+ інструменти

Специфікацію **Agent Skills** опублікували в грудні 2025. За 48 годин її підтримали OpenAI і Microsoft. Сьогодні її читають Claude Code, Codex CLI, Gemini CLI, Cursor, Kiro та ще десятки інструментів.

Для нас це означає одне: скіл можна писати **не під конкретну модель**. Правильно написаний скіл переживе зміну інструменту — а інструменти в цій галузі змінюються швидше, ніж наші проекти.

2

обов'язкові поля frontmatter — і все

32+

інструменти читають цей формат

Мінімальний контракт

```
name: release-notes
  ≤64 символи, lowercase,
  дефіси, збігається з папкою
description: ...
  ≤1024 символи
```

Де лежать командні скіли

У репозиторії, у `.agents/skills/` — нейтральна папка стандарту, а не папка конкретного вендора. Кожен інструмент знає, де шукати.

Валідація

```
npx skills-ref validate ./my-skill
```

ТЕЗИ

- Це слайд про стратегію, не про синтаксис. Головна думка: **ми не хочемо прив'язуватись до одного вендора**.
- Реальність галузі: інструмент, яким ви користуєтесь сьогодні, може бути не найкращим через рік. Скіли — це актив, який має пережити цю зміну.
- Згадати, що в cardona-core-service уже є папка `.agents/skills/` — ми вже зробили перший крок, просто ще не всюди.

ЯКЩО СПИТАЮТЬ

«Тобто скіл гарантовано працюватиме однаково скрізь?» Ні. Стандартизований *пакет*: структура, frontmatter, порядок розкриття. Шляхи встановлення, модель прав і спосіб виклику лишаються різними. Тому наступний слайд — про те, що саме ламає портативність.

Що ламає портативність — і чим це замінити

✗ Прив'язує до одного інструменту

- ◆ Вендорні поля frontmatter:
`context: fork`, `when_to_use`,
`user-invocable`,
`disable-model-invocation`
- ◆ Продуктові дієслова: «відкрий панель X»,
«натисни кнопку Y»
- ◆ Абсолютні шляхи й припущення про конкретний раннер чи структуру папок
- ◆ Форк скіла під кожен інструмент — далі вони роз'їжджаються назавжди

✓ Працює скрізь

- ◆ Умовні формулювання: «якщо інструмент підтримує вкладення — додай скріншот»
- ◆ Тонкі адаптери під конкретний інструмент (`agents/openai.yaml`) — хто не знає формату, той просто ігнорує файл
- ◆ Відносні шляхи всередині пакета скіла
- ◆ Критичні політики продубльовані в `AGENTS.md` — щоб агент бачив їх навіть коли скіл не спрацював

✓ Чеклист: перш ніж назвати скіл портативним — прогнати у двох інструментах

- 1 · `discovery` — скіл видно в каталозі
- 2 · `activation` — тіло вантажиться на реальному промпті
- 3 · `references` — лінковані файли доступні
- 4 · `scripts` — виконуються під реальною моделлю прав
- 5 · `output` — якість збігається з очікуваною

- Найпідступніший пункт — вендорні поля. Вони зручні, і саме тому їх ставлять не думаючи.
- Правило: **хочеш вендорну фічу — винеси її в адаптер, не в SKILL.md.**
- Пункт про AGENTS.md важливий: безпекові й критичні правила не можна ховати тільки в скіл, бо скіл може не спрацювати.
- Чеклист із 5 пунктів — реальна робота на 15 хвилин, не теорія.

ЯКЩО СПИТАЮТЬ

«А наші скіли зараз портативні?» Частково. Структура й progressive disclosure — так. Але є прив'язки до конкретних сабагентів і до нашого харнесу. Це наш план: винести спільну частину в .agents/skills/, а вендорне — в адаптери.

Скіл **impact-analysis**: чотири файли, чотири різні ролі

L2 SKILL.md

4.2 КБ

Тільки маршрутизація для головного асистента: запустити сабагента, передати звіт дослівно, запропонувати запис у Jira.

L3 playbook.md

9.6 КБ

Інструкції для воркера. Головний асистент їх **ніколи не читає** — їх читає сабагент у своєму контексті.

L3 cardona-map.md

14 КБ

Карта розділів беку: шлях у меню, URL, що там перевіряти. Довідка — вантажиться за потреби.

— collect-evidence.mjs

28 КБ

Детермінований збір діфа й доказів. **Нуль токенів** — це просто код, який виконується.

♦ Два правила вартості, які ми записали в CLAUDE.md

1. У SKILL.md — тільки те, що потрібно головному асистенту. Усе для воркера — в `references/playbook.md`. Інакше дорога модель платить за весь плейбук, а сабагент платить за нього вдруге.

2. Детермінована робота — у скрипт, не в модель. Саме для цього існують `permissions-add.mjs`, `section-scaffold.mjs` і `collect-evidence.mjs`.

ТЕЗИ

- Це ілюстрація до попередніх слайдів, не самоціль. Показати, що теорія прикладається до реального коду.
- Найцікавіша цифра: 28 КБ скрипта — це нуль токенів. Якби це робила модель, це була б найдорожча частина скіла.

- Правило №1 — про подвійну оплату. Проговорити повільно, воно неочевидне: контент, покладений не туди, оплачується двічі.
- Сказати чесно: ми прийшли до цих правил не одразу, а після того, як поміряли бюджет.

ЯКЩО СПИТАЮТЬ

«Звіт українською, а спілкування російською — навіщо?» Звіт іде в Jira для мануальних QA, у них робоча мова звітів українська. Це приклад того, що в скіл треба класти й такі домовленості — інакше кожен раз буде по-різному.

П'ять способів зіпсувати хороший скіл

1 Скіл-енциклопедія

Усе в одному SKILL.md «щоб не шукати». Результат: модель платить за все одразу і губиться в тому, що їй зараз не потрібно.

2 Дублювання AGENTS.md

Конвенції, які й так читаються завжди, переписані ще й у скіл. Оплачується двічі, роз'їжджається при першій же зміні.

3 Скіл, який не тригериться

Написаний ідеально, description порожній. Живе в репозиторії, не працює ніколи. Найпоширеніший випадок.

4 Форк під кожен інструмент

Три копії одного скіла під три тули. Через місяць вони різні, і ніхто не знає, яка з них правильна.

5 Важкий зовнішній скіл без перевірки

Поставили «бо корисний» і забули поміряти.

! Наш живий приклад №5

Зовнішній [skill-creator](#) у нашому репозиторії — **11 171 токен** on-trigger. Це найдорожчий скіл у проєкті, і потрібен він рівно тоді, коли ти пишеш новий скіл. Тобто майже ніколи.

Висновок: зовнішні скіли теж треба міряти. Ми повернемось до цієї цифри в блоці про токени.

ТЕЗИ

- Пункт 5 — навмисний місток у блок про токени. Сказати: «запам'ятайте цифру 11 171, ми до неї повернемось».
- Чесність тут важлива: ми самі притягли цей скіл і самі його не поміряли. Це нормально — ненормально не міряти взагалі.

- Пункт 2 — типовий для команд, які щойно почали: люди про всяк випадок дублюють конвенції всюди.

Чат, workflow і агент — різниця в одному питанні

Хто вирішує, який крок буде наступним?

◆ Чат

Вирішує людина. Ти питаєш — модель відповідає. Один хід, один результат.

крок = 1

Добре для: пояснити, згенерувати шматок, обговорити варіант.

◆ Workflow

Вирішує код. Кроки прописані заздалегідь, модель викликається на конкретних етапах.

кроки фіксовані

Добре для: повторюваних конвеєрів, де шлях відомий наперед.

◆ Агент

Вирішує модель. LLM у циклі з інструментами і явною умовою зупинки.

кроки невідомі наперед

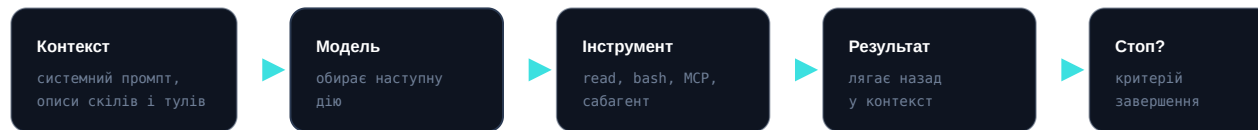
Добре для: задач, де шлях залежить від того, що знайдеться по дорозі.

Практичний висновок: агент коштує дорожче й поводить менш передбачувано. Якщо шлях відомий наперед — робіть workflow або скрипт. Агент виправданий там, де кроки не можна прописати, бо вони залежать від проміжних знахідок.

ТЕЗИ

- Це слайд-фільтр: більшість «агентів», які люди роблять, насправді мали бути workflow.
- Приклад агентної задачі: «знайди, чому падає цей тест». Шлях невідомий — залежить від того, що покаже перший запуск.
- Приклад НЕ агентної: «згенеруй CRUD за шаблоном». Кроки відомі — це скрипт плюс модель на одному етапі.

Що насправді відбувається всередині



ні → наступна ітерація, контекст росте з кожним колом

Кожне коло додає токени. Агент без критерію зупинки — це агент, який зупиниться на ліміті вікна.

Звідси головна теза блоку: **агент — це функція від контексту, а не від промпту**. Керувати агентом означає керувати тим, що лягає в це вікно на кожному колі.

ТЕЗИ

- Провести пальцем по циклу вголос — це те, чого багато хто ніколи не бачив явно.
- Ключове: **результат кожного інструменту лягає назад у контекст**. Прочитав великий файл — він з тобою до кінця сесії.
- Саме тому сабагент з ізольованим контекстом такий цінний: він читає 20 файлів у себе, а назад віддає абзац.
- Червона пунктирна стрілка — місце, де все ламається, якщо немає стоп-умови.

ЯКЩО СПИТАЮТЬ

«А компакція контексту це не вирішує?» Частково. Компакція стискає історію, але стискає і деталі — після неї агент часто «забуває» домовленості. Краще не засмічувати, ніж потім чистити.

Агент і Skill пов'язані у два боки

→ Агент тягне скіли

Скіли — це процедури, якими агент користується. Агент-рев'юер сам вирішує, що зараз доречний `code-style`, а на наступному файлі — `ally-audit`.

```
// агент «reviewer», контекст свій
задача: перевірка MR #482
→ бачить .vue файли
→ підтягує skill ally-audit
→ бачить зміну в store
→ підтягує skill code-style
→ віддає один зведений коментар
```

← Скіл делегує в агента

Зворотний напрямок, який використовують рідше. Скіл лишає собі маршрутизацію, а важку роботу віддає сабагенту з **чистим контекстом і дешевшою моделлю**.

```
// skill «impact-analysis»
головний асистент:
→ Agent(subagent: impact-analysis)
→ сабагент читає 30 файлів у СВОЄМУ вікні
→ повертає готовий звіт
→ головний передає дослівно
```

◆ Навіщо другий напрямок

Це головний важіль економії у великих задачах. Сабагент прочитав 30 файлів — це **30 файлів у його вікні, а не у вашому**. Назад приходить абзац тексту. Головний контекст лишається чистим, і сесія живе далі.

ТЕЗИ

- Більшість знає перший напрямок і не знає другого. Другий — найцінніший.
- Метафора: сабагент — це не «ще один AI», це **ізолюваний контекст із дешевшою моделлю**.
- Порахувати вголос: 30 файлів × ~2000 токенів = 60 000. Назад — 400 токенів звіту. Різниця в 150 разів.

ЯКЩО СПИТАЮТЬ

«А сабагент не втратить контекст задачі?» Втратить — і це фіча. Йому треба передати рівно те, що потрібно для його шматка. Якщо для роботи йому потрібна вся ваша історія — це не той шматок, який варто виносити.

П'ять правил доброго агента

- 1 **Вузька роль.** «Аналізує вплив змін для QA» — так. «Допомагає з кодом» — ні. Роль має вміщатись в одне речення без слова «і».
- 2 **Ізольований контекст.** Агент отримує рівно те, що потрібно для його шматка, і нічого більше.
- 3 **Найдешевша достатня модель.** Класифікація і збір фактів — маленька модель. Складне міркування — велика. «Orus на все» це не якість, це рахунок.
- 4 **Детермінізм — у скрипт.** Те саме правило, що й для скілів. Агент не має рахувати те, що рахує `wc -l`.
- 5 **Явний крок верифікації.** Прогнати тести, перерхитати діф, звірити з критерієм. Без цього агент упевнено віддає неправильне.

♦ Як обирати модель під роль

Класифікація, витяг фактів	найменша
Структурований звіт за шаблоном	середня
Архітектурне рішення, дебаг	найбільша

! Правило верифікації

«Подумай ще раз» — це не верифікація. Верифікація — це **зовнішній факт**: тест упав чи пройшов, лінтер мовчить чи ні, поле в Jira реально перерхиталось після запису.

ТЕЗИ

- Правило 3 — найбільша економія в грошах. Правило 5 — найбільша економія в нервах.
- Розкрити правило 5 на живому прикладі: у нашому скілі root-cause після запису поля в Jira є обов'язкове перерхитування — бо API може віддати 200 і мовчки не зберегти поле.
- Сказати: «якщо агент не може перевірити свою роботу зовнішнім фактом — швидше за все, задача поставлена неправильно».

Шість сабагентів і три різні моделі

САБАГЕНТ	РОЛЬ	МОДЕЛЬ
impact-analysis	звіт для QA з git-діфа	sonnet
section-list	сторінка-список розділу	sonnet
section-form	сторінка-форма розділу	sonnet
write-tests	unit-тести на composables	sonnet
root-cause	класифікація причини бага	fable
backoffice-permissions	права доступу розділу	haiku

◆ Наскрізний сценарій `/ia`

- 1 Скіл вирішує: аналіз не тут, а в сабагенті
- 2 Сабагент читає діф у своєму вікні на sonnet
- 3 Повертає готовий звіт українською
- 4 Головний асистент передає дослівно і питає: писати в Jira?
- 5 Запис у поле, **перечитування для перевірки**, і пропозиція `/root-cause`

Чому запис у Jira лишили в головному контексті: він має нести реальний намір користувача і вміти показати запит на дозвіл. Сабагенту таке не довіряють.

ТЕЗИ

- Головна ілюстрація тут — колонка з моделями. Три різні моделі під три різні класи задач.
- backoffice-permissions на haiku — це майже класифікація, великій моделі там нічого робити.
- Пункт про Jira — важливий патерн безпеки: **дії з побічними ефектами лишаються там, де є людина.**
- Не заглиблюватись у деталі наших розділів — зала не з нашої команди по беку.

ЯКЩО СПИТАЮТЬ

«Скільки часу пішло це побудувати?» Ітеративно, кілька місяців, і половину переписали після першого заміру бюджету. Починати варто з одного скіла, а не з шести агентів.

П'ять агентів, яких не варто робити

1 Агент-універсал

«Помічник по проєкту».
Робить усе потроху й нічого
добре. Контекст роздутий,
бо має бути готовий до
всього.

2 Агент замість скрипта

Модель робить те, що
детерміновано робить код.
Дорожче, повільніше, і
щоразу трохи інакше.

3 «Подумай ще раз»

Самоперевірка без
зовнішнього факту. Модель
упевнено підтверджує
власну помилку.

4 Агент без стоп-умови

Крутиться, поки не впреться
в ліміт вікна. Формально
«працює», по суті — горить
бюджет.

5 Найбільша модель на все

Класифікація на топовій
моделі — це переплата в
десятки разів за задачу, з
якою впрається найменша.

✓ Протиотрута

Одне речення ролі без «і» ·
один вимірюваний результат
· зовнішній критерій
завершення · найдешевша
модель, яка ще
справляється.

ТЕЗИ

- Швидкий слайд, 40 секунд. Не читати всі картки — назвати заголовки, розкрити одну-дві.
- Пункт 3 варто розкрити: це та сама проблема, що й код-рев'ю самого себе.

МСР — єдиний протокол «агент ↔ зовнішня система»

До нього кожна інтеграція писалась окремо під кожен інструмент. Тепер сервер пишеться один раз і його бачать усі агенти, що говорять протоколом.

tools

Дії. Те, що агент може викликати: створити задачу, запустити запит, надіслати повідомлення. Мають побічні ефекти.

resources

Дані для читання.
Адресується URI, читається без побічних ефектів. Саме сюди має йти все, що просто «подивитись».

prompts

Готові шаблони.
Параметризовані сценарії, які сервер пропонує клієнту. Використовуються рідше за перші два.

Транспорти: `stdio` для локальних серверів (запускається як процес поруч) і `Streamable HTTP` для віддалених.

Важливо: МСР не ухвалює рішень. Це руки, не голова. Уся логіка лишається в скілах і агентах.

ТЕЗИ

- Аналогія, яку люблять: МСР — це USB-C для агентів. Один роз'єм замість кабелю під кожен пристрій.
- Найважливіша практична річ на слайді: **розділення tools і resources**. Якщо це читання без наслідків — це resource, і його можна кешувати.
- Не заглиблюватись у транспорти, якщо зала не питає.

Що змінилось у специфікації 2026-07-28

◆ Протокол став stateless

Немає рукописання `initialize` і немає session id. Кожен запит самоописовий: несе версію протоколу, ідентичність клієнта і його можливості. Наслідок — будь-який запит може лягти на будь-який інстанс за балансером, без спільного сховища.

◆ MRTT — багатокрокові запити

Сервер може відповісти `resultType: "input_required"` і доуточнити параметри чи попросити підтвердження **посеред виклику** — замість утримування відкритого стріму.

◆ Маршрутизація за заголовками

Заголовки `Mcp-Method` і `Mcp-Name` дозволяють гейтвею й rate-лімітеру приймати рішення, не розбираючи JSON-тіло.

◆ Кешовані list-результати

Відповіді на list і read несуть `ttlMs` і `cacheScope`. Клієнт перестає перезапитувати те, що не змінилось.

! Deprecated

Roots, Sampling і Logging позначені як застарілі — мінімум 12 місяців до видалення. Якщо ваш сервер на них спирається, є час переїхати.

ТЕЗИ

- **Це опційний слайд.** Пропустити, якщо йдете довго — на розуміння решти доповіді він не впливає.
- Головне, що варто сказати навіть при пропуску: протокол став stateless, і це різко спрощує хостинг власного сервера.
- Deprecation Sampling — важливий для тих, хто планує писати сервер: не закладатись на нього.

Прихована ціна: тули платяться в кожній сесії

✗ Наївний підхід

- ◆ Підключили 6 серверів — це 40-60 визначень тулів у системному промпті **кожної сесії**, навіть якщо ви не викликали жодного
- ◆ Результат виклику проходить через модель **двічі**: прийшов у контекст, потім модель переписала його в наступний виклик
- ◆ Вивантажили 500 рядків з API, щоб узяти з них три поля — усі 500 у вікні назавжди



✓ Рішення, які вже є

- ◆ **Tool search.** Визначення тулів не висять у промпті — агент шукає й підвантажує потрібні на вимогу
- ◆ **Code execution with MCP.** Агент пише код, який викликає тули й фільтрує результат — у контекст повертається тільки підсумок
- ◆ **Точкові запити.** Просити конкретні поля, а не «віддай усе»

На типових пайплайнах це до ~98% економії токенів на роботі з тулами.

Правило, яке з цього впливає: кожен доданий тул — це податок на кожну сесію. П'ять добре названих тулів із чіткими описами корисніші, ніж сорок сирих. Проєктування MCP-сервера починається з питання «які тули тут зайві».

ТЕЗИ

- Це найпрактичніший слайд блоку MCP. Люди підключають сервери й не розуміють, чому контекст закінчується на другому питанні.
- «Двічі через модель» проговорити повільно — це неочевидно.
- Наш живий приклад точкового запиту: у скілі impact-analysis прямо написано «ніколи не запитуй *all з Jira — payload величезний». Це правило народилось із болю.

- Цифру 98% подавати як «до», а не як гарантію — вона з опублікованих кейсів на конкретних пайплайнах.

Свій MCP-сервер як агрегатор

Звичний погляд: MCP — це обгортка над чужим API. Але сервер можна писати й «навпаки»: як **точку збору й роздачі стану** для всього парку агентів.

ДЖЕРЕЛА — пишуть

Агент у репо А
результати рев'ю

Агент у CI
падіння тестів, флейки

Агент у репо В
метрики бандла

Локальні агенти
нотатки, рішення

Ваш MCP-сервер

нормалізація у спільну схему

- ◆ приймає події від будь-якого агента
- ◆ зводить їх у єдину модель даних
- ◆ зберігає з версією схеми

resources → читання накопиченого
tools → запити, зрізи, дії

СПОЖИВАЧІ — читають

Дашборд
той самий сервер як бекенд

Інший агент
«що вже знає команда?»

Звіти й алерти
зрізи за періодом

ТЕЗИ

- Це слайд «розширення кругозору» — приклад свідомо не про наші продукти.
- Ключовий зсув у голові: MCP може бути не тільки *виходом* агента назовні, а й **спільною пам'яттю парку агентів**.
- Найцінніша частина — права стрілка: той самий сервер віддає дані і агенту, і звичайному дашборду. Один контракт, два типи споживачів.
- Приклади, які заходять: зведення флейків з усіх репозиторіїв; спільний реєстр архітектурних рішень; накопичення метрик бандла по проєктах.

ЯКЩО СПИТАЮТЬ

«Це ж просто API з базою, навіщо MCP?» Саме так — усередині це звичайний сервіс. MCP дає стандартний контракт, завдяки якому будь-який агент бачить його без окремої інтеграції, з описами й типами. Ви пишете сервер один раз, а не адаптер під кожен інструмент.

Шість рішень, які треба ухвалити до першого рядка коду

1 Контракт схеми

Дані пишуть різні агенти в різний час. Схема з версією й валідацією на вході — інакше через місяць у вас п'ять форматів одного поля.

2 Ідемпотентність

Агент упав і повторив запис. Потрібен ключ ідемпотентності, інакше у звітах будуть дублі, яких ніхто не помітить.

3 Авторизація і скоупи

Кожен агент має свою ідентичність і свій набір прав. CI-агент пише, дашборд тільки читає. Не один токен на всіх.

4 TTL і кеш

Позначайте `ttlMs` на `list` і `read`. Найдешевший запит — той, який клієнт не зробив.

5 Бюджет тулів

Кожен тул — податок на кожну сесію кожного агента. П'ять добре названих корисніші за сорок сирих. Читання виносьте в `resources`.

6 Спостережуваність

Хто, що і коли записав. Коли агент запише дурницю — а він запише — вам потрібно буде знайти, який саме і коли.

Головна пастка: писати MCP-сервер як «ще одне API». Різниця в тому, що **ваш клієнт — модель**. Вона читає назви й описи тулів як документацію і помиляється там, де людина перепитала б. Назви й описи тут — не косметика, а частина контракту.

ТЕЗИ

- Пункт 2 — з практики: агенти падають і ретрають значно частіше, ніж класичні клієнти.
- Пункт 5 — місток назад до слайда про приховану ціну. Нагадати.
- Фінальна плашка — головна думка блоку: **ваш клієнт не людина, а модель**. Проговорити обов'язково.

- Якщо є час — спитати залу, у кого є задача, яка лягла б на такий агрегатор. Зазвичай хтось згадує флейки або метрики збірок.

Що підключено — і що свідомо не через модель

◆ MCP-сервери в роботі

Atlassian	читання полів задачі, запис Impact analysis і Root cause
Sentry	помилки й трейси до root-cause аналізу
Figma	дизайн-контекст під час верстки
Slack	сповіщення й пошук по обговореннях
Playwright	перевірка сценаріїв у браузері

◆ Hooks — детермінований шар поруч

```
SessionStart
  sync-core-claude.mjs  симлінки скілів
  docs-inject.mjs      індекс вікі в контекст
  docs-hooks-install.mjs  git-хуки

Stop
  docs-guard.mjs      нагадати оновити доку
  root-cause-guard.mjs  нагадати про поле в J
```

Те, що має статись **завжди**, не доручають моделі. Хук виконається, навіть якщо агент про нього «забув» — і це коштує нуль токенів.

ТЕЗИ

- Головна думка слайда — права колонка, а не ліва. Hooks недооцінені.
- Сформулювати правило: **«має статись завжди» = хук, «залежить від ситуації» = скіл або агент.**
- docs-guard питає рівно один раз і тільки після push — це важлива деталь UX: хук, який нудить, вимикають на другий день.
- Згадати kill switch CARDONA_DOCS_GUARD=0 — будь-яка автоматизація має мати вимикач, інакше їй не довіряють.

Далі — технологія, яку ми **не використовуємо**

Чесна рамка, щоб не було непорозумінь. Наступні три слайди — не пропозиція щось переписувати і не план на спринт. Це нова технологія, яка з'явилась цього місяця. Я хочу, щоб ви знали, що вона існує, розуміли, яку проблему вона закриває, і могли оцінити її самі — **якщо** вона доживе до статусу стандарту.



Проблема, яка є вже зараз

Агент постійно ухвалює дрібні судження:

- ◆ Ця команда небезпечна чи ні?
- ◆ Який із 30 скілів зараз доречний?
- ◆ Чи релевантний цей файл до задачі?
- ◆ Чи підтверджує джерело це твердження?

За кожне таке судження платиться повноцінною генерацією тексту. Судження дешеві по суті — але дорогі по механіці.



Ідея, яку варто знати

Замість того, щоб просити модель *написати текст* про своє рішення — попросити її **повернути саме рішення**: один з варіантів, оцінку або ймовірність. Без генерації тексту.

Це окремий клас моделей. Швидших і дешевших на два порядки — але вужчих.

ТЕЗИ

- **Рамку проговорити вголос майже дослівно.** Інакше хтось піде з думкою, що ми це впроваджуємо.
- Мотивація блоку: галузь рухається швидко, і корисно розуміти напрямок руху, навіть коли не біжиш першим.
- Ліва картка важлива сама по собі — вона описує реальну проблему нашого сьогодення харнесу, незалежно від того, чим її вирішувати.

Jev: модель, що повертає рішення, а не текст

«System One»-модель від TypeSafe AI, вересень 2026. Немає авторегресії — тобто немає посимвольної генерації. Є типізована відповідь на поставлені питання.

choice

Який із варіантів? Повертає обраний + розподіл ймовірностей по всіх.

70–500_{ms}

затримка end-to-end

\$0.042_m

вхідні токени

score

Де це на шкалі? Повертає числову оцінку за заданою рубрикою.

\$0

вихід — немає циклу генерації

noul

Це правда? Повертає ймовірність від 0 до 1.

1

виклик — усі питання паралельно

Головна властивість і головне обмеження в одному реченні: модель **не може повернути невалідний тип** — але цілком може повернути **валідне неправильне значення**. Плюс до кожної відповіді йде калібрована впевненість, за якою можна ставити поріг.

ТЕЗИ

- Пояснити «без авторегресії» просто: звичайна модель будує відповідь по токenu, тут одразу віддається рішення. Звідси і швидкість, і нульова ціна виходу.
- Калібрована впевненість — недооцінена частина. Вона дозволяє будувати логіку «якщо впевненість нижче 0.8 — передай великій моделі».
- Фразу про валідний тип / неправильне значення проговорити повільно. Це запобіжник від надмірних очікувань.
- Ціни назвати як порядок величини, а не як точну цифру — вони змінюються.

ЯКЩО СПИТАЮТЬ

«Це ж просто класифікатор?» По суті — так, але з двома відмінностями: він загальнопризначений (рубрику задаєте ви в запиті, без донавчання) і навчений саме на калібрування ймовірностей, а не тільки на точність.

П'ять патернів із публічних кейсів

1 Роутер здібностей

Замість тримати описи всіх скілів у промпті — локальний шорт-лист по BM25, одне **choice** - питання, максимум одна підказка. «Утриматись» — повноцінний варіант відповіді.

p95 406 ms з кешем

221 ms на повторі

2 Guardrail перед tool-call

Перед виконанням команди — 4 питання за ~250 мс: чи незворотна дія, чи поза задачею, чи змінює дані, який масштаб.

17 000 викликів

42 зупинки

3 Реранкінг результатів

Оцінити знайдене **до** того, як це прочитає велика модель. У опублікованому кейсі юридичного пошуку точність top-1 зросла з 5% до 18%.

4 Скринінг prompt injection

По кілька **noul** -питань на кожную завантажену сторінку: релевантна, має докази, суперечить передумові, схожа на ін'єкцію. Підкинутий шкідливий текст отримав 0.99 і був відкинутий.

5 Роутинг моделі

Оцінити складність і намір запиту — і вибрати найдешевшу модель, яка впорається. Доступно через шлюзи на кшталт Vercel AI Gateway.

? Гіпотези для нас — саме гіпотези

- ◆ Вибір скла замість опису всіх скілів у кожній сесії
- ◆ Гейт «чи змінює цей коміт контракт сторінки» перед оновленням вікі
- ◆ Фільтр файлів до того, як сабагент їх прочитає

- Патерн 2 — найпереконливіший: 42 зупинки на 17 000 викликів, тобто він не заважає, але ловить.
- Патерн 3 — найближчий до нашого болю: агент читає файли, які йому не потрібні. Відсіювання до читання — саме те, чого бракує.
- Остання картка — обов'язково наголосити слово «гіпотези». Це не беклог.
- Цифри з кейсів подавати як чужі виміри, не як наші обіцянки.

Де воно ламається — і за яких умов ми б його взяли

✗ Задokumentовані обмеження

- ◆ **Буквальне читання.** Відповідає на ваші слова, а не на ваш намір. Формулювання питання критичне
- ◆ **Математика й підрахунки.** Бачить форму, не значення
- ◆ **Дати.** Порівнює як текст, а не як дати
- ◆ **Multi-hop.** На складених питаннях точність помітно падає
- ◆ **Великий шумний стан.** Розсіюється так само, як великі моделі
- ◆ **Adversarial-текст.** Її можна «повести» підкинутим вмістом

✓ Наші критерії «коли беремо»

- ◆ **Стабільний SDK** і доступ без вейтліста
- ◆ **Прийнятний privacy-контур.** Промпти йдуть у зовнішній сервіс — для нашого коду це окреме рішення, а не дрібниця
- ◆ **Вимірюваний виграш** саме на нашому бюджеті, а не в чужому кейсі
- ◆ **Статус стандарту, а не хайпу.** Технології цього класу з'являються швидше, ніж встигають дозріти

Поки що: спостерігаємо. Без витрат, без переписування, без обіцянок.

Підсумок блоку: **це фільтр і роутер, а не заміна міркування.** Там, де потрібне міркування, дешева модель лишається дешевою моделлю.

ТЕЗИ

- Права колонка важливіша за ліву — вона показує, як ми взагалі ухвалюємо рішення про нові інструменти.
- Пункт про privacy проговорити окремо: наш код у зовнішньому сервісі — це питання не до фронтенду, а до компанії.
- Останній рядок — головний винос блоку. Сказати його як фінальну крапку і йти далі.

ЯКЩО СПИТАЮТЬ

«А є щось таке, що можна хостити локально?» З'явилися відкриті репродукції ваг, але зрілість у них інша. Якщо задача реально стоїть — починати варто з вимірювання, скільки ми взагалі платимо за дрібні судження. Може виявиться, що гра не варта свічок.

Контекст — це бюджет, а не безліміт

Вікно в 200 тисяч токенів створює ілюзію, що можна не думати. Але якість відповіді падає **задовго** до формального ліміту — цей ефект називають **context rot**.

Чим більше в контексті всього, тим важче моделі втримати те, що справді важливо. Зайвий файл не просто коштує грошей — він **погіршує рішення**.

Мета оптимізації — не «менше токенів». Мета — менше сміття. Скорочувати треба те, що не впливає на рішення. Усе інше залишати.

✗ Що засмічує

- ◆ Повні дампи API замість потрібних полів
- ◆ Файли, прочитані «про всяк випадок»
- ◆ Довгі скіли, що спрацювали не до речі
- ◆ Описи 40 тулів, з яких ви юзаєте два

✓ Що працює

- ◆ Зовнішня пам'ять замість історії в вікні
- ◆ Сабагенти з власним контекстом
- ◆ Точкові запити й фільтрація до читання
- ◆ Скрипти замість міркувань

ТЕЗИ

- Розкрити context rot побутово: «це як шукати потрібний рядок у файлі на 10 тисяч рядків — технічно все там є, практично ви його не знайдете».
- Головна фраза: **зайвий файл не просто коштує грошей, він погіршує рішення**. Це те, чого не розуміють ті, хто дивиться тільки на рахунок.

- Перехід до наступного слайда: «а тепер найнудніша, але найкорисніша частина — як це поміряти».

Спочатку виміряй. Оптимізувати без цифр — самообман

Універсальний підхід: розділіть вартість обв'язки на два принципово різні класи.

A

Always-on — платиться в кожній сесії

Описи всіх скілів і агентів, кореневий AGENTS.md / CLAUDE.md, правила, описи команд, payload стартових хуків. Навіть якщо ви нічого не викликали.

CLAUDE.md		1 078
описи скілів і агентів		1 405
docs-inject (індекс вікі)		366
описи команд		42

Разом: 2 891 токен у кожній сесії, до першого слова.

B

On-trigger — платиться при виклику

Тіло SKILL.md, тіло агента і файли references, які вони реально читають. Тут можна дозволити собі обсяг — але треба знати, який.

skill-creator (зовнішній)		11 171
impact-analysis		3 088
root-cause		2 196
update-docs		1 116
write-tests		196

Один зовнішній скіл дорожчий за всі наші разом.

ТЕЗИ

- Розділення always-on / on-trigger — головна методична ідея. Без нього люди оптимізують не те.
- Цифри реальні, зняті інструментом `claude-budget.mjs` у нашому `core-service`. Оцінка — символи/4, тому важливі не абсолютні, а порівняння.
- Ось і обіцяна цифра 11 171 зі слайда про антипатерни. Нагадати залі.
- `write-tests` на 196 токенів — приклад ідеального скіла: усе винесено в `references`, у тілі лише маршрут.

ЯКЩО СПИТАЮТЬ

«Символи поділити на чотири — це ж дуже грубо» Так, $\pm 15\%$. Але нам і не потрібна точність: потрібне коректне порівняння до і після, а на цій метриці воно коректне.

Три зміни за годину роботи — і ось що вийшло

◆ Що саме зробили

1

Прибрали зовнішній `skill-creator` — інструмент для написання скілів, потрібен раз на місяць

2

Розділили `root-cause`: у SKILL.md лишилась маршрутизація, тіло процедури переїхало в `references/playbook.md`

3

Стиснули три найдовші `always-on` описи, не втративши тригерні слова

```
# робочий цикл
node scripts/claude-budget.mjs --baseline
# ... вносимо зміни ...
node scripts/claude-budget.mjs --compare
```

`always-on`

2 891

→ стало

2 683

`root-cause`

2 196

→ стало

477

35 190

22 092

–37%

було, токенів

стало

–13 098 токенів

Жодного рядка інструкцій не втрачено — усе, що прибрали з гарячого шляху, лежить поруч і читається за потреби. Це не економія на якості. Це прибирання.

ТЕЗИ

- Це кульмінація блоку. Не поспішати, дати цифрам повисіти.
- Наголосити: жоден інструктивний рядок не видалений. `root-cause` просто переїхав на рівень 3.
- Чесно сказати: найбільший шматок економії — це видалення зовнішнього скіла, тобто просто прибирання того, що ми самі притягли й не поміряли.
- Донести головне: **–37% за годину роботи — це те, що може зробити будь-хто в залі у своєму проєкті вже завтра.**

ЯКЩО СПИТАЮТЬ

«А якість не впала?» Перевіряється не вірою, а прогоном тих самих задач до і після. У випадку root-cause процедура ідентична — змінилось лише те, коли вона читається.

Сім важелів — і що не можна чіпати

1 Progressive disclosure

Три рівні скіла.
Найбільший ефект
на найменших
зусиллях.

2 Ізольований сабагент

Він читає 30 файлів
у себе, вам віддає
абзац.

3 Модель під роль

Класифікація —
найменша модель.
Різниця в десятки
разів.

4 Детермінізм у скрипт

Найдешевші токени
— ті, яких немає.

5 Prompt caching

Стабільний префікс
контексту — не
мініяйте його без
потреби.

6 Точковий запит

Конкретні поля
замість «віддай
усе».

7 Зовнішня пам'ять

Вікі й файли
замість роздутої
історії в вікні.

✗ Не економити на

Критеріях
приймання.
Прикладах входу-
виходу. Кроці
верифікації.

Живий приклад важеля №6 з нашого скіла: `getJiraIssue` викликається зі списком конкретних полів, і в інструкції прямо написано «ніколи не `*all` — цей payload величезний». Одне речення в скілі, яке економить тисячі токенів у кожному виклику.

ТЕЗИ

- Пройти швидко по номерах, зупинитись на 4 і 7.
- Важіль 5 неочевидний: якщо ви щоразу мініяєте початок контексту, кеш не працює. Стабільний префікс — безкоштовна економія.
- Червона картка — найважливіша. Економія на критеріях приймання повертається переробками, які дорожчі за будь-які токени.

- Фінальна плашка показує, що правило народжується з болю і фіксується одним реченням у скілі.

Агенту потрібна карта, а не територія

Один із наших репозиторіїв: **1 002 файли, ~616 500 слів**. Це більше, ніж будь-яке вікно контексту, і значно більше, ніж варто читати.

Питання не в тому, як засунути проєкт у модель.
Питання в тому, **як дати їй орієнтуватись, не читаючи все**.

файли в репозиторії

слів — близько 800 тис. токенів

1 Граф коду

Механічні зв'язки: хто кого імпортує, що на що впливає. Відповідає на питання **«де»**.

2 LLM wiki

Смисли, яких немає в коді: інваріанти, контракти, «чому саме так». Відповідає на питання **«чому»**.

3 Пошук на вимогу

греп і читання конкретного файлу. Відповідає на питання **«як саме»**.

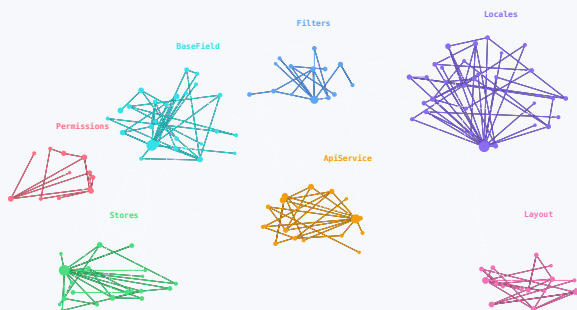
Вони не конкурують. Граф каже, куди дивитись; вікі каже, що там важливо; пошук дає точні деталі.

ТЕЗИ

- Цифра 616 500 слів — це приблизно 800 тисяч токенів, тобто в 4 рази більше за вікно. Сказати вголос, вона вражає.
- Три стратегії й три питання — «де / чому / як саме» — це каркас, який легко запам'ятати.

- Найчастіша помилка: намагатись замінити одне іншим. Вікі не замінює граф, граф не замінює читання коду.

Граф файлів: 2 717 вузлів, 3 772 зв'язки, 337 спільнот



♦ Що дає граф на виході

- ◆ **Спільноти** — автоматичні кластери, кожен стає точкою входу в підсистему
- ◆ **God nodes** — найзв'язаніші абстракції. У нас: `BaseField` (46 ребер), `useUserStore` (22)
- ◆ **Цикли імпортів** — знайшлися такі, про які ніхто не знав
- ◆ **Несподівані зв'язки** — «а чому старий authCore тягне сучасний стор?»
- ◆ **Прогалини** — 1 280 майже ізольованих вузлів: або мертвий код, або недоописані місця

! Несподіваний побічний ефект

Найбільшу користь від графа отримали **люди, а не агенти**. Список циклів імпортів і god nodes — це готовий беклог технічного боргу, якого ніхто до того не бачив цілком.

- Пояснити коротко, як буде утворюватися: AST-розбір плюс семантичний шар, на виході `graph.json`, інтерактивний `graph.html` і текстовий `GRAPH_REPORT.md`.
- Для агента найцінніший саме текстовий звіт: спільноти — це дешеві точки входу замість сліпого `grep` по 1000 файлів.
- Історія про цикли імпортів заходить найкраще. У нас знайшлися цикли на 5 файлів через `filterConfig` → `baseField` → `index` → `baseList`.
- Чесно: 1 280 ізольованих вузлів — це і мертвий код, і просто межа точності інструменту. Не подавати як діагноз.

ЯКЩО СПИТАЮТЬ

«Скільки коштує побудувати граф?» Структурний прохід — безкоштовний, це чистий AST. Семантичний шар на великому репозиторії вже платний, і на 1000 файлів його варто запускати точково по підпапках.

LLM wiki: три шари, три операції, одне правило

Три шари

```
src/      сирі джерела
  ↓
knowledge/ вікі: смисли
  ↓
AGENTS.md схема + скіли
```

Кожен шар щільніший за попередній. Агент іде згори вниз і зупиняється, щойно знайшов відповідь.

Три операції

- ◆ **ingest** — оновити сторінку після змін у коді
- ◆ **query** — читати вікі **перед** тим, як відкрити джерело
- ◆ **lint** — бити посилання, сироти, застарілі сторінки

Без третьої операції вікі помирає за два місяці. Це не опція.

Правило сторінки

Сторінка несе те, **що не виводиться дешево з коду**: інваріанти, контракти, підводні камені, «чому саме так», порядок викликів.

Не список полів і не сигнатури. Переказ файлу дорожчий за сам файл і старіє миттєво.

Якщо сторінка — це переказ, її не має бути. Ліміт ~2 КБ.

Свіжість — по хешу вмісту, а не по mtime

```
--- frontmatter сторінки ---
title: ApiService
source: src/services/api/index.ts
source_hash: 963e4a053157 ← ось це
updated: 2026-07-27
```

Реальний біль, який це вирішило:

ми починали з mtime. Після кожного `yarn install` або перемикання гілки половина вікі оголошувалась застарілою — і люди перестали вірити індикатору. Хеш вмісту прибрав проблему повністю.

ТЕЗИ

- Правило сторінки — головне, що варто винести із цього слайда. Більшість документації для AI — це саме переказ коду, тобто марна витрата.

- Операція **query** найнедооціненіша: вона вчить агента спочатку читати сторінку на 2 КБ, а не файл на 30 КБ.
- Історія з mtime — практична порада, яку зала може забрати собі одразу. Це та деталь, на якій спотикаються всі.
- Наголосити на lint: вікі без ліну — це вікі, якій через квартал ніхто не довіряє.

Документація оновлюється сама — і чи варто воно того



Незакомічені правки ніколи не нудять. Вимикач: `CARDONA_DOCS_GUARD=0` — будь-яка автоматизація має мати вимикач, інакше їй не довіряють.

✓ Що реально працює

- ◆ Онбординг: нова людина читає 20 сторінок замість 1000 файлів
- ◆ Агент відповідає «де це лежить» без сліпого пошуку
- ◆ Знання команди перестало бути в голові двох людей

~ Що це коштує

- ◆ Дисципліна: сторінки треба ревіювати, як код
- ◆ Спокуса писати документацію заради документації
- ◆ Робочий механізм — це ще й скрипти, які хтось підтримує

! Де межа

- ◆ На проєкті до сотні файлів це не окупається — простіше читати код
- ◆ Без ліну й ревію вікі гниє й починає шкодити
- ◆ Сторінки-перекази гірші за їх відсутність

ТЕЗИ

- Конвеєр показати швидко, головне на слайді — три нижні картки.
- Чесність тут критична: якщо продавати це як срібну кулю, першому ж скептику буде що заперечити.
- Найсильніший аргумент «за» — онбординг. Він вимірюваний і його всі відчували.
- Сказати прямо: **на маленькому проєкті цього робити не треба**. Це чесно й підвищує довіру до решти доповіді.

ЯКЩО СПИТАЮТЬ

«Хто пише сторінки — людина чи агент?» Агент пропонує, людина рев'ює в тому ж MR, що й код. Сторінка без рев'ю — це припущення моделі, а не знання команди.

Один апдейт ядра — і скіли з'явилися у всіх панелей

```
cardona-core-service/.claude/*  канонічні копії
    ↓ npm
node_modules/cardona-core-service
    ↓ SessionStart hook
scripts/sync-core-claude.mjs
    ↓ відносні симлінки
cardona/.claude/skills/*
marbella-panel/.claude/skills/*
compostela-panel/.claude/skills/*
alaro-panel/.claude/skills/*
```

◆ Чому симлінки, а не копії

Інструмент не сканує `node_modules`.
Симлінк робить скіл видимим,
лишаючи одне джерело правди.

◆ Проєкт завжди перемагає

Реальна папка в панелі **затіняє**
симлінк. Потрібна своя версія скіла —
просто створи її поруч, і вона виграє.

Скрипт ідемпотентний і ніколи не
роняє процес — щоб не ламати
`yarn install` і CI.

ТЕЗИ

- **Опційний слайд.** Пропустити, якщо часу мало — це наша внутрішня інфраструктура.
- Універсальна ідея, яку варто озвучити навіть при пропуску: **у скілів має бути одне джерело правди й механізм роздачі**. Копіаста між репозиторіями не масштабується.
- «Проєкт затіняє ядро» — гарний патерн, знайомий усім по CSS і по конфігах.

П'ять кроків, які можна зробити завтра

- 1 AGENTS.md на одну сторінку.** Конвенції, команди, чого не робити. Одна сторінка, не десять.
- 2 Вимірй бюджет.** Розділи always-on і on-trigger. Побачиш, за що платиш у кожній сесії.
- 3 Один скіл на найнуднішу рутину.** Не шість. Один, який ти робиш руками щотижня.
- 4 Винеси детермінізм у скрипт.** Усе, що «порахуй / знайди / розпарси».
- 5 Помірй різницю.** Baseline → зміна → порівняння. Без цього це не інженерія, а віра.

→ Куди рухаємось ми

- ◆ **Універсальні скіли** за відкритим стандартом — щоб не залежати від одного інструменту
- ◆ **Свій MCP-сервер-агрегатор** — спільна пам'ять для парку агентів
- ◆ **Жев під наглядом** — спостерігаємо, не впроваджуємо

Якщо забути все інше — пам'ятайте одне: агент це функція від контексту. Керуйте тим, що в нього потрапляє, і 80% проблем зникнуть самі.

ТЕЗИ

- Пункт 3 проговорити з натиском: люди зазвичай намагаються побудувати всю систему одразу і кидають.
- Права нижня плашка — фінальна думка доповіді. Сказати її повільно й зробити паузу.
- Запропонувати допомогу: хто хоче зробити свій перший скіл — покажу на своєму прикладі за 20 хвилин.

Питання?

Розберемо будь-що детальніше

◆ Стандарти й документація

Agent Skills open standard ·
Claude Docs: skill authoring
best practices · MCP
specification 2026-07-28 ·
AGENTS.md

◆ Інженерні матеріали

Anthropic: effective context
engineering for AI agents ·
Code execution with MCP ·
TypeSafe AI: Jev docs і
публічні кейси (jev-skillful, pi-
warden)

◆ Наші репозиторії

cardona-core-service ·
cardona · compostela-panel ·
marbella-panel · alaro-panel
— `.claude/`,
`knowledge/`, `scripts/`

Слайди й нотатки спікера — у роздатці (Ctrl+P). Питання, які не встигнемо, — пишіть, розберемо окремо.

ОЧІКУВАНІ ПИТАННЯ

«З чого почати саме мені?» З вимірювання. Поки не знаєш, за що платиш, будь-яка оптимізація — вгадування.

«Скільки часу це забирає в тижні?» Після того як механізм стоїть — майже нуль. Основна вартість у перших двох тижнях, поки формулюєш правила.

«А якщо інструмент завтра зміниться?» Саме тому блок про портативність. Скіли за відкритим стандартом переживуть зміну, самописна обв'язка — ні.

«Чи не робить це нас залежними від AI?» Скіли й вікі корисні людям так само: це записані процедури й пояснення, яких раніше не було ніде.

ЯКЩО ЧАСУ ЛИШИЛОСЬ БАГАТО

- Показати живцем `node scripts/claude-budget.mjs` на екрані — це найсильніша демонстрація.
- Або відкрити `graph.html` і покрутити граф.